

CT520 –Algoritmos e Estruturas de Dados

EN-AEL – 5º Ano

Ordenar por ordem crescente o seguinte array usando cada um dos algoritmos de ordenação abaixo indicados.

$A=[5,10,3,1,7]$

- 1) Bubble Sort
- 2) Insertion Sort
- 3) Selection Sort
- 4) Radix Sort
- 5) Quick Sort
- 6) Merge Sort
- 7) Heap Sort

Resolução:

1) Bubble Sort

Estado inicial:

[5, 10, 3, 1, 7]

Passagem 1

- 5 e 10 → não troca
[5, 10, 3, 1, 7]
- 10 e 3 → troca
[5, 3, 10, 1, 7]
- 10 e 1 → troca
[5, 3, 1, 10, 7]
- 10 e 7 → troca
[5, 3, 1, 7, 10]

Passagem 2

- 5 e 3 → troca
[3, 5, 1, 7, 10]
- 5 e 1 → troca
[3, 1, 5, 7, 10]
- 5 e 7 → não troca
[3, 1, 5, 7, 10]

Passagem 3

- 3 e 1 → troca
[1, 3, 5, 7, 10]
- 3 e 5 → não troca

Resultado final:

[1, 3, 5, 7, 10]

2) Insertion Sort

Estado inicial:

[5, 10, 3, 1, 7]

A parte ordenada começa com o primeiro elemento.

Passo 1: inserir 10 na parte ordenada [5]

- $10 > 5 \rightarrow$ fica no fim

[5, 10, 3, 1, 7]

Passo 2: inserir 3 na parte ordenada [5,10]

- $3 < 10 \rightarrow$ 10 desloca
- $3 < 5 \rightarrow$ 5 desloca
- entra 3

[3, 5, 10, 1, 7]

Passo 3: inserir 1 na parte ordenada [3,5,10]

- $1 < 10 \rightarrow$ 10 desloca
- $1 < 5 \rightarrow$ 5 desloca
- $1 < 3 \rightarrow$ 3 desloca
- entra 1

[1, 3, 5, 10, 7]

Passo 4: inserir 7 na parte ordenada [1,3,5,10]

- $7 < 10 \rightarrow$ 10 desloca
- $7 > 5 \rightarrow$ entra depois do 5

[1, 3, 5, 7, 10]

Resultado final:

[1, 3, 5, 7, 10]

3) Selection Sort

Estado inicial:

[5, 10, 3, 1, 7]

Passo 1

- menor de [5,10,3,1,7] é 1
- troca com o primeiro elemento

[1, 10, 3, 5, 7]

Passo 2

- menor de [10,3,5,7] é 3
- troca com a posição 1

[1, 3, 10, 5, 7]

Passo 3

- menor de [10,5,7] é 5
- troca com a posição 2

[1, 3, 5, 10, 7]

Passo 4

- menor de [10,7] é 7
- troca com a posição 3

[1, 3, 5, 7, 10]

Resultado final:

[1, 3, 5, 7, 10]

4) Radix Sort

Estado inicial:

[5, 10, 3, 1, 7]

O maior número é 10, então existem até **2 dígitos**.

Passagem 1 — unidades

- 5 → unidade 5
 - 10 → unidade 0
 - 3 → unidade 3
 - 1 → unidade 1
 - 7 → unidade 7
-
- 0: [10]
 - 1: [1]
 - 3: [3]
 - 5: [5]
 - 7: [7]

Juntando:

[10, 1, 3, 5, 7]

Passagem 2 — dezenas

- 10 → dezena 1
 - 1 → dezena 0
 - 3 → dezena 0
 - 5 → dezena 0
 - 7 → dezena 0
-
- 0: [1, 3, 5, 7]
 - 1: [10]

Juntando:

[1, 3, 5, 7, 10]

Resultado final:

[1, 3, 5, 7, 10]

5) Quick Sort

0	1	2	3	4	Índices
5	10	3	1	7	Posição inicial
5	10	3	1	7	Início 1ª Iteração
5	10	3	1	7	
1	10	3	5	7	
1	3	10	5	7	Fim 1ª Iteração
1	3	10	5	7	Início 2ª Iteração
1	3	10	5	7	Fim 2ª Iteração
1	3	10	5	7	Início 3ª Iteração
1	3	10	5	7	
1	3	5	10	7	Fim 3ª Iteração
1	3	5	10	7	Início 4ª Iteração
1	3	5	10	7	
1	3	5	10	7	
1	3	5	7	10	Fim 4ª Iteração

Pivô

Esq

Dta

6) Merge Sort

Estado inicial:

[5, 10, 3, 1, 7]

Divisão

Divide em duas partes:

- esquerda: [5, 10, 3]
- direita: [1, 7]

Dividir [5,10,3]

- [5,10] e [3]

Dividir [5,10]

- [5] e [10]

Agora começa a junção:

- juntar [5] e [10] → [5,10]
- juntar [5,10] e [3]:
 - compara 5 com 3 → entra 3
 - depois entram 5 e 10Resultado: [3,5,10]

Dividir [1,7]

- [1] e [7]
- juntar → [1,7]

Junção final

Juntar:

- [3,5,10]
- [1,7]

Comparações:

- 3 vs 1 → entra 1
- 3 vs 7 → entra 3
- 5 vs 7 → entra 5
- 10 vs 7 → entra 7
- sobra 10

Resultado:

[1, 3, 5, 7, 10]

Resultado final:

[1, 3, 5, 7, 10]

7) Heap Sort

Estado inicial:

[5, 10, 3, 1, 7]

1. Construir o Max Heap

Começando de baixo para cima, obtemos:

[10, 7, 3, 1, 5]

Aqui, o maior valor está na raiz.

2. Ordenação

Passo 1

Troca a raiz com o último:

[5, 7, 3, 1, 10]

Reheapify nos primeiros 4 elementos:

[7, 5, 3, 1, 10]

Passo 2

Troca a raiz com o último da parte não ordenada:

[1, 5, 3, 7, 10]

Reheapify nos primeiros 3:

[5, 1, 3, 7, 10]

Passo 3

Troca a raiz com o último da parte não ordenada:

[3, 1, 5, 7, 10]

Reheapify nos primeiros 2:

[3, 1, 5, 7, 10]

Passo 4

Troca a raiz com o último da parte não ordenada:

[1, 3, 5, 7, 10]

Resultado final:

[1, 3, 5, 7, 10]